

Nama : Achmad Pradita Dwi Firmansyah
NIM : 254107020130
Kelas / Absen : TI – 1G / 01

JOBSHEET XII

Double Linked List

12.1 Percobaan 1: Operasi Penambahan pada Double Linked List

12.1.1 Kode Program

Class Mahasiswa01

```
package Jobsheet12;

public class Mahasiswa01 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    Mahasiswa01(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    void tampil() {
        System.out.println(
            "NIM : " + nim +
            "\nNama : " + nama +
            "\nKelas : " + kelas +
            "\nIPK : " + ipk
        );
    }
}
```

Class Node01

```
package Jobsheet12;

public class Node01 {
    Mahasiswa01 data;
```

```

Node01 prev;
Node01 next;

Node01(Mahasiswa01 data) {
    this.data = data;
    this.prev = null;
    this.next = null;
}
}

```

Class DoubleLinkedList01

```
package Jobsheet12;
```

```

public class DoubleLinkedList01 {
    Node01 head;
    Node01 tail;

    DoubleLinkedList01() {
        head = null;
        tail = null;
    }

    boolean isEmpty() {
        return head == null;
    }

    void addFirst(Mahasiswa01 data) {
        Node01 newNode = new Node01(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }
}

```

```

void addLast(Mahasiswa01 data) {
    Node01 newNode = new Node01(data);
    if (isEmpty()) {
        head = tail = newNode;
    } else {
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    }
}

```

```

void insertAfter(String keyNim, Mahasiswa01 data) {
    Node01 current = head;
    while (current != null && !current.data.nim.equals(keyNim)) {
        current = current.next;
    }
    if (current == null) {
        System.out.println("Data dengan NIM " + keyNim + " tidak ditemukan.");
        return;
    }
}

```

```

Node01 newNode = new Node01(data);

```

```

// jika current adalah tail, node baru ditambahkan di akhir

```

```

if (current == tail) {
    newNode.prev = current;
    current.next = newNode;
    tail = newNode;
} else { // node baru disisipkan di tengah
    newNode.prev = current;
    newNode.next = current.next;
    current.next.prev = newNode;
    current.next = newNode;
}
System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
}

```

```

void print() {
    if (isEmpty()) {
        System.out.println("Linked list masih kosong.");
        return;
    }

    Node01 current = head;
    while (current != null) {
        current.data.tampil();
        current = current.next;
    }
}
}

```

Class DoubleLinkedList01Main

```
package Jobsheet12;
```

```
import java.util.Scanner;
```

```

public class DoubleLinkedListMain01 {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        DoubleLinkedList01 list = new DoubleLinkedList01();
        int pilihan;
        do {
            System.out.println("\n===== MENU DOUBLE LINKED LIST =====");
            System.out.println("1. Tambah data di awal");
            System.out.println("2. Tambah data di akhir");
            System.out.println("3. Sisipkan data di tengah");
            System.out.println("4. Hapus data di awal");
            System.out.println("5. Hapus data di akhir");
            System.out.println("6. Tampilkan data");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu : ");
            pilihan = scan.nextInt();
            scan.nextLine();
            switch (pilihan) {

```

```

case 1:
    Mahasiswa01 mhsAwal = inputMahasiswa(scan);
    list.addFirst(mhsAwal);
    break;
case 2:
    Mahasiswa01 mhsAkhir = inputMahasiswa(scan);
    list.addLast(mhsAkhir);
    break;
case 3:
    System.out.print("Masukkan NIM yang dicari : ");
    String keyNim = scan.nextLine();
    System.out.println("Masukkan data baru: ");
    Mahasiswa01 dataBaru = inputMahasiswa(scan);
    list.insertAfter(keyNim, dataBaru);
    break;
case 4:
    list.removeFirst();
    break;
case 5:
    list.removeLast();
    break;
case 6:
    list.print();
    break;
case 0:
    System.out.println("Program selesai.");
    break;
default:
    System.out.println("Menu tidak valid.");
    break;
}
} while (pilihan != 0);
scan.close();
}
}
}

```

12.1.2 Verifikasi Hasil Percobaan

```
===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 2
Masukkan NIM : 123005
Masukkan Nama : Harry
Masukkan Kelas : 1A
Masukkan IPK : 3,76

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 3
Masukkan NIM yang dicari : 123005
Masukkan data baru:
Masukkan NIM : 123010
Masukkan Nama : Potter
Masukkan Kelas : 1B
Masukkan IPK : 3,55
Data berhasil disisipkan setelah NIM 123005

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 6
NIM : 123005
Nama : Harry
Kelas : 1A
IPK : 3.76
NIM : 123010
Nama : Potter
Kelas : 1B
IPK : 3.55
```

12.1.3 Pertanyaan

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!

Pada Single Linked List hanya menggunakan pointer next , namun pada Double Linked List menggunakan 2 pointer next dan prev, sehingga pada Double Linked List bisa mengakses node dari depan ataupun dari belakang

2. Perhatikan class **Node**, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi

masing-masing atribut tersebut pada proses traversal dan manipulasi node!

Next : Berfungsi untuk menunjuk ke node berikutnya, dalam proses traversal ini memungkinkan untuk maju ke node berikutnya, dan dalam manipulasi node ini akan mengarahkan ke node berikutnya.

Prev : Berfungsi untuk menunjuk ke node sebelumnya, dalam proses traversal ini memungkinkan untuk mundur ke node sebelumnya, dan dalam manipulasi node ini akan mengarahkan ke node sebelumnya.

3. Perhatikan konstruktor pada class **DoubleLinkedList**. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!

Pada kondisi awal linked list berarti data masih kosong sehingga didalam konstruktor di inisialisasi head dan tail sama dengan null

4. Perhatikan potongan kode berikut:

```
if (isEmpty()) {  
    head = tail = newNode;  
}
```

Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?

Karena hanya ada 1 node dan node tersebut menjadi satu-satunya node yang paling depan dan paling belakang sehingga head dan tail harus menunjuk node tersebut.

5. Modifikasi method **print()** agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data pada linked list!

```
void print() {  
    if (isEmpty()) {  
        System.out.println("Linked list masih kosong.");  
        return;  
    }  
  
    Node01 current = head;  
    while (current != null) {  
        current.data.tampil();  
        current = current.next;  
    }  
}
```

6. Modifikasi kode program dengan menambahkan method **printReverse()** untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

```
void printReverse() {  
    if (isEmpty()) {  
        System.out.println("Linked list masih kosong.");  
        return;  
    }  
}
```

```

    }

    Node01 current = tail;
    while (current != null) {
        current.data.tampil();
        current = current.prev;
    }
}

```

12.2 Percobaan 2: Operasi Penghapusan pada Double Linked List

12.2.1 Kode Program

Class Mahasiswa01

```

package Jobsheet12;

public class Mahasiswa01 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    Mahasiswa01(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    void tampil() {
        System.out.println(
            "NIM : " + nim +
            "\nNama : " + nama +
            "\nKelas : " + kelas +
            "\nIPK : " + ipk
        );
    }
}

```

Class Node01

```

package Jobsheet12;

public class Node01 {
    Mahasiswa01 data;
    Node01 prev;
    Node01 next;

    Node01(Mahasiswa01 data) {
        this.data = data;
    }
}

```



```

        this.prev = null;
        this.next = null;
    }
}

```

Class DoubleLinkedList01

```
package Jobsheet12;
```

```

public class DoubleLinkedList01 {
    Node01 head;
    Node01 tail;

    DoubleLinkedList01() {
        head = null;
        tail = null;
    }

    boolean isEmpty() {
        return head == null;
    }

    void addFirst(Mahasiswa01 data) {
        Node01 newNode = new Node01(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }

    void addLast(Mahasiswa01 data) {
        Node01 newNode = new Node01(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    void insertAfter(String keyNim, Mahasiswa01 data) {
        Node01 current = head;
        while (current != null && !current.data.nim.equals(keyNim)) {
            current = current.next;
        }
    }
}

```

```

    if (current == null) {
        System.out.println("Data dengan NIM " + keyNim + " tidak ditemukan.");
        return;
    }

    Node01 newNode = new Node01(data);

    // jika current adalah tail, node baru ditambahkan di akhir
    if (current == tail) {
        newNode.prev = current;
        current.next = newNode;
        tail = newNode;
    } else { // node baru disisipkan di tengah
        newNode.prev = current;
        newNode.next = current.next;
        current.next.prev = newNode;
        current.next = newNode;
    }
    System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
}

void print() {
    if (isEmpty()) {
        System.out.println("Linked list masih kosong.");
        return;
    }

    Node01 current = head;
    while (current != null) {
        current.data.tampil();
        current = current.next;
    }
}

void printReverse() {
    if (isEmpty()) {
        System.out.println("Linked list masih kosong.");
        return;
    }

    Node01 current = tail;
    while (current != null) {
        current.data.tampil();
        current = current.prev;
    }
}

void removeFirst() {

```

```

        if (isEmpty()) {
            System.out.println("Linked List kosong.");
            return;
        }

        System.out.println("Data berhasil dihapus.");
        head.data.tampil();

        if (head == tail) {
            head = tail = null;
        } else {
            head = head.next;
            head.prev = null;
        }
    }

    void removeLast() {
        if (isEmpty()) {
            System.out.println("Linked List kosong.");
            return;
        }

        System.out.println("Date berhasil dihapus.");
        tail.data.tampil();

        if (head == tail) {
            head = tail = null;
        } else {
            tail = tail.prev;
            tail.next = null;
        }
    }
}

```

Class DoubleLinkedList01Main

```
package Jobsheet12;
```

```
import java.util.Scanner;
```

```

public class DoubleLinkedListMain01 {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        DoubleLinkedList01 list = new DoubleLinkedList01();
        int pilihan;
        do {
            System.out.println("\n===== MENU DOUBLE LINKED LIST =====");
            System.out.println("1. Tambah data di awal");
            System.out.println("2. Tambah data di akhir");

```

```

        System.out.println("3. Sisipkan data di tengah");
        System.out.println("4. Hapus data di awal");
        System.out.println("5. Hapus data di akhir");
        System.out.println("6. Tampilkan data");
        System.out.println("0. Keluar");
        System.out.print("Pilih menu : ");
        pilihan = scan.nextInt();
        scan.nextLine();
        switch (pilihan) {
            case 1:
                Mahasiswa01 mhsAwal = inputMahasiswa(scan);
                list.addFirst(mhsAwal);
                break;
            case 2:
                Mahasiswa01 mhsAkhir = inputMahasiswa(scan);
                list.addLast(mhsAkhir);
                break;
            case 3:
                System.out.print("Masukkan NIM yang dicari : ");
                String keyNim = scan.nextLine();
                System.out.println("Masukkan data baru: ");
                Mahasiswa01 dataBaru = inputMahasiswa(scan);
                list.insertAfter(keyNim, dataBaru);
                break;
            case 4:
                list.removeFirst();
                break;
            case 5:
                list.removeLast();
                break;
            case 6:
                list.print();
                break;
            case 0:
                System.out.println("Program selesai.");
                break;
            default:
                System.out.println("Menu tidak valid.");
                break;
        }
    } while (pilihan != 0);
    scan.close();
}

public static Mahasiswa01 inputMahasiswa(Scanner scan) {
    System.out.print("Masukkan NIM : ");
    String nim = scan.nextLine();
    System.out.print("Masukkan Nama : ");

```

```

        String nama = scan.nextLine();
        System.out.print("Masukkan Kelas : ");
        String kelas = scan.nextLine();
        System.out.print("Masukkan IPK  : ");
        double ipk = scan.nextDouble();
        scan.nextLine();
        return new Mahasiswa01(nim, nama, kelas, ipk);
    }
}

```

12.2.2 Verifikasi Hasil Percobaan

```

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 5
Date berhasil dihapus.
NIM    : 123010
Nama   : Potter
Kelas : 1B
IPK    : 3.55

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 6
NIM    : 123005
Nama   : Harry
Kelas : 1A
IPK    : 3.76

```

12.2.3 Pertanyaan Percobaan

1. Perhatikan potongan kode berikut pada method `removeFirst()`:

```

head = head.next;
head.prev = null;

```

Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!

Fungsi statement `head = head.next;` untuk mengubah head yang awalnya pada node paling depan, dipindahkan ke node berikutnya.

Kemudian `Head.prev = null`; Pointer `prev` dari `head` di akan arahkan ke `null` bukan mengarah ke node yang ingin dihapus

2. Modifikasi method `removeFirst()` dan `removeLast()` agar program menampilkan data yang berhasil dihapus!

```
void removeFirst() {  
    if (isEmpty()) {  
        System.out.println("Linked List kosong.");  
        return;  
    }
```

```
    System.out.println("Data berhasil dihapus.");  
    head.data.tampil();
```

```
    if (head == tail) {  
        head = tail = null;  
    } else {  
        head = head.next;  
        head.prev = null;  
    }  
}
```

```
void removeLast() {  
    if (isEmpty()) {  
        System.out.println("Linked List kosong.");  
        return;  
    }
```

```
    System.out.println("Date berhasil dihapus.");  
    tail.data.tampil();
```

```
    if (head == tail) {  
        head = tail = null;  
    } else {  
        tail = tail.prev;  
        tail.next = null;  
    }  
}
```